
DESCRIPTION COMPLETE DU PROJET : TESTEUR DE MEMOIRE

Équipe : Kevin E. Jean & Nguyen Ta Anh Quoc

Date : 14 avril 2026

Cours : Objet Connecté - Épreuve Finale

Table de matière

Résumé de la solution	1
Architecture du système	2
Spécifications fonctionnelles	3
Conception technique	3
Liste des composants (Bill of Materials)	4
Schéma d'architecture	5
Calendrier des livrables	5

Résumé de la solution

Description du jeu :

Le jeu commence par l'apparition d'un ou plusieurs points clignotants à des positions aléatoires (le nombre correspond au niveau de difficulté), que le joueur doit suivre en les dirigeant à l'aide d'un *d-pad*. L'interface comprend des boutons pour démarrer, réinitialiser et terminer la partie. Aussi, il affiche l'état du jeu, le score, et un paramètre de difficulté. Un bouton physique permet d'afficher un dessin enregistré, puis un autre enregistre un dessin à la fois. Ce jeu est inspiré du "*memory leap frog de Lego Party*".

À faire :

Configurer le Wi-Fi, développer une interface utilisateur, mettre en place un service de communication et automatiser le lancement des scripts (Wi-Fi, backend, frontend).

Conception :

Pour ce faire, le wifi sera configuré sur le serveur 'rasp08' à l'aide de '*NetworkManager*' et '*dnsmasq.d*'. Ensuite, l'interface sera développée avec les langages de programmation suivants : HTML, CSS et Javascript. Puis, le service de communication utilisé sera HTTP REST, parce qu'il facilitera l'échange de données entre l'interface web et le '*backend*'. Finalement, le lancement des scripts sera effectué en tant que service '*systemd*' de type '*one-shot*'.

Besoin :

L'utilisateur ne sait pas comment tester si son mémoire est efficace et il voudrait que cela soit divertissant. On a créé un jeu qui teste la réaction et le mémoire du joueur. Il contrôlerait le jeu avec un joystick sur une matrice 8x8 pour voir s'il souvient les positions des petits boules rouges sur la matrice. Pour qu'il ne se sente pas seul, notre équipe a ajouté un deuxième joueur pour le rendre plus divertissant. On croit aussi que l'esprit compétitif rendra l'utilisateur à mieux performer.

Architecture du système

Infrastructure Réseau :

Wi-Fi Local : Le wifi sera host sur le serveur local rasp08 à l'aide de NetworkManager et dnsmasq.

Protocole : La communication s'effectuera en HTTP REST pour garantir un échange de données.

Composantes logicielles et matérielles :

Machine : Raspberry Pi4

Backend : Python Flask

Frontend: HTML/CSS/JS

Spécifications fonctionnelles

Flux de données (Senseur → Serveur) :

À chaque seconde, la led RGB sera lue et son état sera envoyé au serveur. De plus, durant une partie, un d-pad et une matrix 8x8 sera lue à chaque seconde. Ensuite, lorsqu'une partie n'est pas en cours, deux boutons seront lus chaque seconde, un enregistre l'état de la matrix, l'autre affiche cette état (led allumer/éteint).

Contrôle (Serveur → Objet) :

Depuis l'interface web, durant une partie les led de la matrix 8x8 pourront être allumé ou éteint, puis à la suite d'une tentative, le buzzer et la led RGB sera activer/désactiver.

Interaction en temps réel :

Il y a une copie de la matrix 8x8 sur l'interface web, en cliquant sur un de ses boutons, la led correspondante à la matrix 8x8 sera allumée/éteint. Trois boutons contrôlent l'état de la led RGB au click.

Conception technique

Structure de données

Interface web → Serveur :

```
{  
  "game_state": "start",  
  "difficulty": "Easy"  
}
```

Serveur → Interface web :

```
{  
  "player1_score": 0,  
  "player2_score": 0,  
  "state": "Connected"  
}
```

Logique et Gestion des erreurs

Algorithme de démarrage : Connexion Wi-Fi -> Validation serveur -> Allumage de la LED -> Boucle de lecture de capteur.

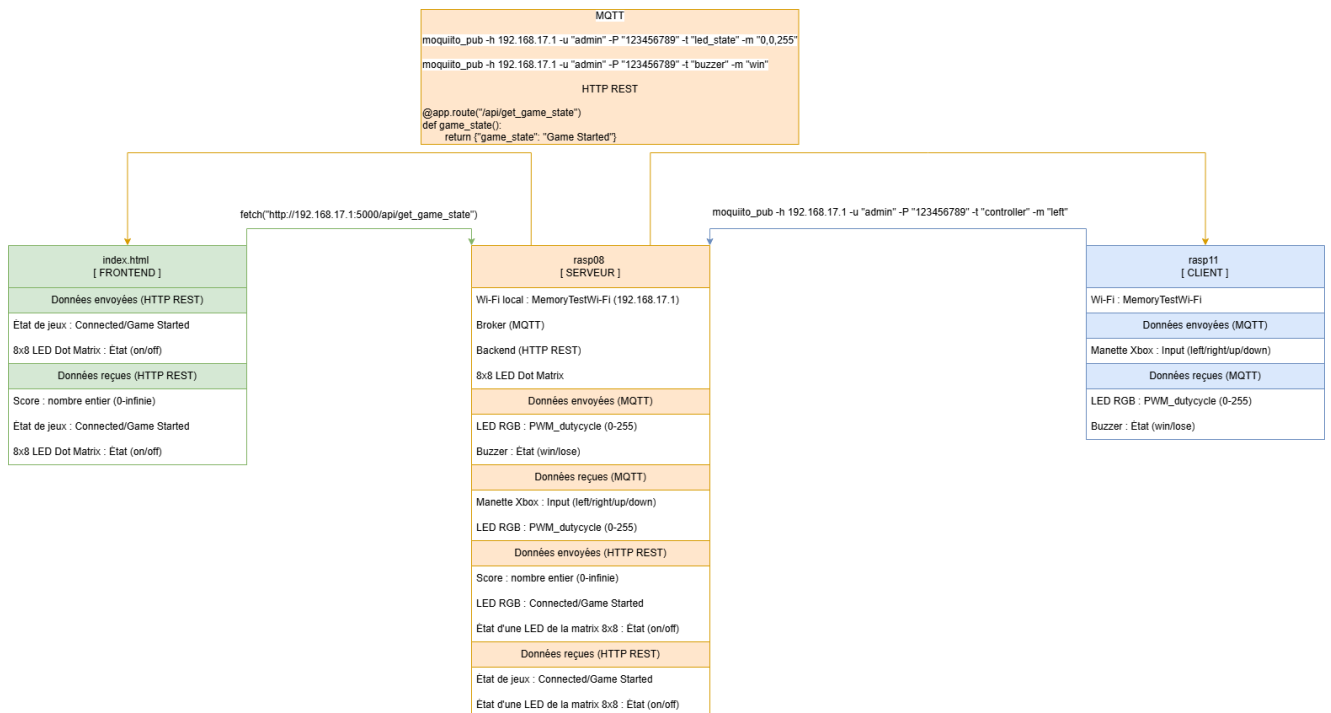
Robustesse : En cas de perte de signal Wi-Fi, l'objet [Il clignotera la LED de statut (jeu interruptif) pour avertir l'utilisateur].

Liste des composants (Bill of Materials)

Composant	Quantité	Rôle
Raspberry Pi4	2	Hébergement du serveur et mqtt
LED RGB	1	Indicateur visuel de l'état du jeu

8x8 Led Dot Matrix	1	Visuel mouvement du jeu
Buzzer	1	Indicateur sonore de l'état du jeu
Joystick	2	Contrôleur du jeu
Bouton Push	2	Contrôle l'affichage de la matrix 8x8
Fils M-F	14	Fils pour connecter les composantes
Fils M-M	3	Fils pour connecter les composantes
Fils F-F	2	Fils pour connecter les composantes

Schéma d'architecture



Calendrier des livrables

Livable 1 Fondations et Architecture (24 avril) :

- Objectif : Remettre un plan de structure pour le projet finale d'ido.
- Contenu : Le document Word qui contient les exigences du professeur.

Livable 2 Démo 1 - Connectivité (1er mai) :

- Objectif : Créer la connexion entre le frontend et le backend, commencer à implémenter les contrôles du joystick et faire le state du jeu. Implémenter le MQQT
- Contenu : Le code du backend (python) avec le HTTP REST et le html (frontend).

Livable 3 Logique du jeu (8 mai) :

- Objectif : Créer la logique du jeu et connecter le joystick pour que le jeu communique avec la led 8x8.
- Contenu : Le code du backend (python) + la logique du jeu + le code de la manette + le html (frontend).

Livable 4 La robustesse et gestion d'erreurs (15 mai) :

- Objectif : Assurer que toute fonctionne et gérer les exceptions qui apparaissent pendant le jeu.
- Contenu : Le code du backend (python) avec les gestions d'erreurs.